Itch: A package manager for Linux From Scratch

Shayan Nezami

January 18, 2026

Contents

1 Introduction

1.1 Background

In this section, I will set out some key terms and definitions, and explain some crucial concepts, knowledge of which form prerequisites to the rest of the write-up. This is intended mostly towards readers will little knowledge about modern Linux systems.

1.1.1 Linux

Linux, while sometimes referred to as an operating system, like macOS and Windows, is in fact just a kernel. A **kernel** is, in simple terms, the core of an operating system, dealing with the CPU, and allocating memory to processes, among other essential operations. However, on its own, it does not provide a functional system, and is certainly not an operating system itself [1].

An operating system is a collection of software that provides a usable system to an end user. Therefore, the Linux kernel is often distributed along with large amounts of other software, in the form of a Linux **distribution**. There are hundreds, if not thousands, of such distributions, examples including Ubuntu, Debian, and Arch Linux. When I refer to a **Linux system**, this is not in reference to the Linux kernel, but rather a full operating system built on top of the kernel.

1.1.2 Package managers

All computer systems need software to function, so there must be a mechanism for users to install software. At its core, **software** is simply code, that is **compiled** ("translated") to a binary format that can be executed by the processor. Installing software, in its most basic form, just consists of copying those binary files to a location where the system recognises them as executable programs.

For Windows systems, most software comes with a dedicated installer (and uninstaller), which carries out this process for the application. However, this is very rarely the case on Linux systems. Software is often released as just source code, and the user is responsible for the installation.

Manual installation is not particularly difficult (the code simply needs to be downloaded, compiled, tested, then copied into place), however this can get tedious when repeated for hundreds of programs. Furthermore, this makes uninstallation very difficult, as there is no way to know which files were installed by which program, and dependency management becomes near impossible.

Dependencies are software or libraries that are required for a program to be installed. There are also **conflicts** between software, where two programs cannot be simultaneously installed on a system. Often, these occur between versions of packages.

For example, where p denotes a package, p_a may require p_c with version ≥ 3.0, and p_b may conflict with p_c versions ≥ 3.5. This means that for p_a, p_b, p_c to be simultaneously installed, p_c must have version v, where $3.0 \leq v < 3.5$.

Therefore, one of the defining features of a Linux distribution is the package manager they include. **Package managers** can perform all the tasks above, and more. Very often they have access to a repository of trusted and popular software, so the user can install, uninstall, and update software with just one command. For example, the following command in Arch Linux installs the web browser, Firefox:

```
1    # pacman -S firefox
```

Here, pacman - the package manager for Arch Linux [2] - gets the *package* for Firefox from its public repository, and then copies the required files into place, keeping records of what files it installed and where. This incredible ease of use is a massive draw towards Linux, and specifically existing Linux distributions.

1.1.3 Linux From Scratch

Linux From Scratch (LFS) is a project that provides an online, continuously updated, book with instructions on how to build a Linux system yourself, compiling and installing all software manually from source code [3], and not using any of the provided distributions - the vast majority of which come with some form of installer.

1.2 Motivation

Having recently installed a Linux From Scratch system, there is a clear gap for a package manager, which is not provided in the book. This is intentional, to bring the user's attention to compiling and installing the base software from scratch [4]. However, this is not sustainable if the system is to become usable long-term.

An advanced, practically usable system will at some point need complex software packages, including but certainly not limited to graphical toolchains, programming frameworks, and desktop environments. These come with many dependencies, shared libraries, and version conflicts.

Even without such large packages, as a system grows, so will the number of packages (even smaller ones) it has, which feed into the same issues. There are also problems associated with upgrading software, where all files relating to the old version need to be cleanly uninstalled, and replaced with the new version, without breaking the dependencies of any other currently installed software [4]. It is not sustainable to continue without a well-defined package manager at this point.

This drives users into three main groups:

1. They create a basic, personal package manager to suit their needs.
2. They install another distribution's package manager, such as Gentoo's Portage, or Arch's pacman [5].
3. They leave their LFS system as-is after installation, never using it for any practical purpose, rather leaving it as a completed educational experience.

All three of these come with significant, non-trivial downsides.

In the first case, these package managers tend to be simple and limited in scope, and while their creation does provide a good educational experience, they are often not capable of fully supporting a growing system.

Plus, programming ability is not a prerequisite to building an LFS system, so, while a minority, there are users who are capable of installing such a system, but not creating a package manager for it. While there are no formal studies on the proportion of Linux users who are not programmers, online discussions seem to suggest the existence of a notable minority [6].

The second case has the benefit of providing a highly reliable, popular, and documented package manager, often with all the user's needed features and more. However, there are two key downsides here.

Firstly, this in effect turns the user's LFS system into a version of the distribution providing the package manager, eliminates many of the purposes of LFS, which is about creating and maintaining your own

system, with full control over all software installed. The sheer complexity of these package managers removes such control from the users, and while it does often produce a working system, the user has in effect just followed a very long path to installing that distribution.

Secondly, and perhaps most crucially, these package managers have been specifically designed for their own distribution, and an LFS system is by definition different. These systems may handle certain cases differently, or have different directory structures, among other subtle differences. Since the package manager has not been made for, and is not commonly used with, LFS systems, there tends to be a lack of support, with a very small user base that can help with any issues that will eventually arise.

For these reasons, I came to the decision to write a package manager, dedicated to Linux From Scratch systems, addressing all the shortfalls identified.

2 Design

2.1 Non-technical aims

I will start by defining some fundamental aims of the package manager that will be produced. Many will seem trivial, but it is important to set these out to guide the technical design decisions.

1. It must be targetted towards LFS systems. While it should ideally be usable on most Linux systems, decisions should be guided by what is needed on an LFS system, and what the LFS user base would prefer.
2. The compilation ("building") of software should be left to the user, meaning precompiled binaries should never be distributed. Linux From Scratch is based on the principles of compiling ones own software, and this is a key requirement if a package manager is to be used by LFS users.
3. Users should be able to patch and modify source code as they desire prior to installation, with the package manager placing no restriction on this.
4. The compiled software should be stored as *packages* of some form for future use, so that the user does not have to rebuild software to reinstall it. Note that this does not defy Aim 2, since these packages will have been compiled by the user themselves, configured as they desire, rather than externally compiled, then distributed.
5. There should be no central server the package manager needs to interact with to function. It should be able to run wholly locally, with no limitations.
6. It must provide a method for full and clean uninstallation, removing all traces of the software, while not affecting any other programs.
7. Dependencies and conflicts must be tracked and managed appropriately, as a key purpose of this project.
8. It must not simply be a derivative of an existing package manager - there must be something unique about it, or a non-trivial reason to use it instead of another existing package manager. This has been partially addressed in Section 1.2, but it should be continuously thought about.

I chose the name *Itch* for this package manager to reflect the importance of Aim 1, since it is being built for Linux From *Scratch*.

2.2 Package maintenance

Through my research of existing package managers, I have identified three promising methods employed by package managers to install and subsequently uninstall packages. These generally all provide some method to track what files are installed, and where, so that they can be removed on uninstallation.

These are generally independent of the method for dependency resolution, which will be discussed in Section 2.3.

2.2.1 Modifying destdir

This method is used by many package managers made for commerical Linux distributions, such as pacman [2]. This is a very simple method, consisting of configuring the package to be installed in the correct, final, location, but modifying the DESTDIR environment variable upon installation, so that the package is instead placed in a "fake" location where it will not function properly [7].

From there, an archiving utility, often tar, is used to create an archive of the files in this fake directory. This archive is then unpacked in the correct installation location, installing the software as normal. However, now the archive still exits, and within it, it holds a copy of every file that was installed, and where they were placed. This makes uninstallation very simple, as these files are simply iterated over and deleted [8].

2.2.2 Using unionfs

This is a unique approach to package management, and follows the same basic principles as the fake installation location method, but differs greatly in implementation [9].

The current file system is accessed read only, and a new, empty, file system is created, which can be written to. A union between these two is made using the unionfs utility. Therefore, when the package is installed to the union, the union will contain the new state of the system, with the new package installed, and the previously empty file system will contain all of the installed files.

These can then be stored in some format, providing all of the files installed by the package for easy uninstallation. Once the user is happy, the union file system can be copied to the main file system (that was previously accessed read only). This allows easy rollbacks in case of an error mid-install.

The main benefit this has over the DESTDIR method is that files are **forced** to be installed in the empty file system, while a poorly packaged program may ignore the DESTDIR environment variable. However, this is rare, and seems to be more of a theoretical advantage rather than a practical one.

2.2.3 Functional package managers

Functional package managers have been gaining popularity in recent years, and they are based on the principle that all configuration and instructions given to packages must be stored in one central file.

This file is then hashed, and that hash corresponds to the unique state of the system [10]. This is very useful for creating reproducible builds, and makes debugging much simpler since a configuration can be recreated on another device with ease.

While this is a fascinating concept, I made the decision not to research it much further, since these package managers do not fit within my fundamental aims for *Itch*, which is to complement a Linux From Scratch system, simplifying tasks. This would be introducing a whole new paradigm, and users would be

much better served installing an existing, developed, functional package manager if they want a system like this, such as Nix.

2.2.4 Conclusion

Although the unionfs method was fascinating from a technical point of view, I decided to use the DESTDIR method over unionfs. The main reasons were that it is much better documented in case any problems arise, it is much simpler to understand and debug, particularly for users of the program who should ideally understand how it works in full, and the unionfs method did not provide any significant advantages over the DESTDIR method, despite the added complexity.

2.3 Dependency resolution

Now, I must consider the problem of dependency resolution. Each package can depend on, or conflict with, certain other packages, meaning certain packages must be installed with each other, while other packages cannot coexist on a system. To make this more complex, these are most often bounded by versions. See Section 1.1.2 for a deeper explanation.

This means that package managers must come with a method to *resolve* dependencies, meaning to find what packages - and in what versions - need to be installed on the system for a new package to be added.

Dependency resolution is an NP-complete problem [11], meaning in practical terms that it cannot reliably be solved "quickly" (there is no known polynomial time solution). Therefore, shortcuts and optimisations are often taking to make it tractable.

2.3.1 Maintaining a curated repository

The vast majority of popular Linux distributions employ this method, where the maintainers of the distribution maintain an online repository, containing one version of each package available, ensuring that those versions all work together. This greatly simplifies the problem of dependency resolution, since package versions are no longer a concern.

Another reason this method is very commonly used in more mainstream distributions is that the package set becomes universal among systems of the same version, making problems easier to diagnose and debug.

Moreover, this fits very well with the philosophies of some distributions. For example, Arch Linux operates on a rolling-release model, always using the very newest software, so this curated repository only contains the newest usable builds [12]. On the opposite end of the spectrum, Debian focuses on stability, so a curated selection of tested, mature packages enables the creation of very robust systems [13].

However, this does not at all fit within the Linux From Scratch philosophy, which is about users creating their own system as they please. *Itch* is simply intended to be a tool to help users with this, and therefore it should not limit the range of software users can install, or curate package repositories based on release version. This choice should be fully in the hands of the user.

Furthermore, since this relies on an online repository of curated software, it violates Aim 5.

2.3.2 Holding a package database

Another method consists of holding a database with the required dependency information for a number of packages (available in many versions). This differs from the curated repository method, in which one

version of each package is stored which will work with each other.

This allows the user to choose what packages they install, and with what version. Gentoo operates a similar system, using an *ebuild* repository, which contains *ebuild* files with the metadata of packages [14] - most of which have multiple available versions.

However, it does require than an online database to be maintained, violating Aim 5. While this can be mitigated, by allowing users to build their own database (keeping the database in a human-readable format), and by allowing the file to be downloaded once and repeatedly used, meaning the package manager does not need to interact with an online sever every time it runs, it still does violate the spirit of the aim, preventing the package manager being from functioning wholly locally.

2.3.3 Using placeholders

The problem with not holding any form of package database is that the system has no information about packages that could satisfy dependencies. Consider a system (using the notation $p_x - y$ to refer to package x, version y) where, the installed packages are $\{p_a - 3.0,\ p_b - 1.2\}$, and $p_c - 2.0$ is to be installed. However, $p_c - 2.0$ requires $p_a - v > 3.5$ and $p_d - v \leq 1.0$. Now, the system does not know about what packages exist within those ranges, and whether they depend on any other packages, or conflict with any existing packages. This makes dependency management very difficult.

In this case, placeholders can be used, meaning the resolver would simply output the version ranges that a satisfying package must fit into. However, this is a one-way relationship. Where the set S is a satisfying set of packages, and S_p output of the program involving placeholders,

$$S \implies S_p, \quad S_p \not\Longrightarrow S. \tag{1}$$

This is because the dependency information of those packages in the range is not known, nor is the actual package versions within the range.

This does not result in a useful package manager. For large packages with hundreds of (often nested) dependencies, it creates a very arduous path for the user.

Imagine a package $p_e - 4.2.1$ is to be installed, and the package manager identifies that it requires $p_e - 2.0.0 < v \leq 3.2.0$. Trying to install that, the package manager can only then inform the user that it requires p_f. However, the version of p_f the user chose to install ends up conflicting with $p_b - 2.0$, which is installed on the system. The system will have no method to detect these in advance, while a system with a package database would instantly output a satisfying arrangement, if one is available.

Very often, this method will be more time-consuming and frustrating than handling the dependencies yourself, or finding a satisfying arrangement online.

2.3.4 Conclusion

This is a tough situation, where either Aim 5 must be (at least partially) defied, or the dependency resolver produced will not be practically usable.

I can immediately remove the first option, of maintaining a curated repository, since that goes too far against the purpose of a Linux From Scratch package manager (Aim 1), and in effect creates a new distribution. The third option of using placeholders is also not reasonable, since it creates a borderline unusable program.

Therefore, I have no option but to hold a package database, containing just the dependency information for packages. This will be in a form that can be downloaded once, and extended by users. This database

can start with data scraped from other package manager databases, and extended with user contributions.

However, since this does go against the concept I originally had for the package manager, I have decided it would be best to split the package *installer* (the installation and uninstallation components) and the dependency resolver into separate projects. They will still work with each other as a unified package management system, but they should also be capable of working independently, and in conjunction with other package management tools.

Besides the obvious benefit of the package installer component not relying on an external database, this also better follows the Unix philosophy, where each program should complete just one task well [15]. It is also more useful for users who only require one half of the system, and use a different method for either dependency resolution or package installation and removal.

3 Programming the package installer

3.1 Basic installation

The principles of building an installing software can be boiled down to the following steps: (patch), configure, build, (test), install. Since *Itch* is focused on ensuring users still compiling their own code (Aim 2), and allowing patching as desired (Aim 3), it will not interfere with any of these steps until *install*.

To make this clear to the user, I created procedures for *configure*, and *install*:

```
install_prefix="$HOME/prog/school/epq/dest/"

configure() {
    echo "configure as normal, ensuring the following is added:"
    echo "    --prefix=$install_prefix"
}

build() {
    echo "make as normal"
}
```

This also reminds the user to specify the install prefix - the location the software will be once fully installed - to the configure command. It is currently set to a location within my home directory to allow for testing without risk of damaging my system, but this will be changed to /usr (the default prefix for LFS systems), with a configuration file allowing it to be changed, once the development is complete.

The code below performs the installation, with a method similar to that described in Section 2.2.1.

```
temp_dir="/tmp/itch/build"

install() {
    rm -rf "$temp_dir" # in case not properly cleaned before
    mkdir -p $temp_dir

    make DESTDIR="$temp_dir" install # install in temp dir

    findname

    sudo mkdir -p "$pkgpath"

    sudo tar czf "$itchpkgpath" -C "$temp_dir$install_prefix" .
```

```
    installpkg
}

installpkg() { # existing tarball in pkgpath
    cd $install_prefix

    # finds existing files that will be overwritten
    for file in $(tar tzf "$itchpkgpath" | grep -v '/$'); do
        if [ -e "$file" ]; then
            printf "Overwrites '%s'\n" "${file#./}"
        fi
    done

    read -p "Proceed? (yes/NO): " confirmation

    if [ "$confirmation" = "yes" ] || [ "$confirmation" = "y" ]; then
        tar xf "$itchpkgpath"
    else
        echo "Aborted!"
    fi

    rm -rf "$temp_dir"
}
```

The following process is followed by the code above to install packages:

1. A temporary installation directory is created in /tmp/itch/build, and the package is installed there first. This directory is placed in /tmp as it is "made available for programs that require temporary files" [16].
2. The tar archive [17] (a gzipped tarball) of the files to be installed is stored in /var/lib/itch/pkgs/{package name}/{package name}.itchpkg. This both allows the package to be reinstalled without recompilation (Aim 4), and holds the data about files created during installation. It is located in /var as "/var contains variable data files" [16], and these files are updated while the program runs.
3. The files to be installed are compared with any files already installed, finding the files that will be overwritten, so the user can cancel the installation if that would be problematic. The for loop for this was heavily inspired by qpkg's [8] verify function [18].
4. The tarball is unpacked in the final installation directory, completing the installation, and the temporary build directory is removed.

The following procedure determines the name of the package being installed, by either using the name of the current directory, or the parent directory if currently inside a build directory. Since this is not fully robust, the user has the chance to override its output.

```
findname() {
    pkgname=$(basename "$(pwd)")

    # tries to find package name (either current dir or parent if in build dir)
    if [ "$pkgname" = "build" ]; then
        pkgname=$(basename "$(dirname "$(pwd)")")
    fi
```

```

    echo "Enter package name (blank for [$pkgname]):"
    read input

    if [ "$input" != "" ]; then
        pkgname="$input"
    fi
}
```

A switch statement calls the required procedure based on the flags the user passes into the program.

```
case "$1" in
    "-c")
        configure ;;
    "-b")
        build ;;
    "-i")
        if [ -z "$2" ]; then
            install
        else
            getname "$@"
            installpkg
        fi ;;
    *)
        echo "Error!"
        ;;
esac
```

The installpkg procedure is called directly if the name of a package that has already been compiled by the user is specified, skipping the building stage. In this case, the getname procedure is called, setting variable based on the name of the package provided, and ensuring it is valid.

```
getname() {
    pkgname="$2"

    pkgpath="/var/lib/itch/pkgs/$pkgname"
    itchpkgpath="$pkgpath/$pkgname.itchpkg"

    if [ ! -f "$itchpkgpath" ]; then
        echo "Error!"
        exit
    fi
}
```

3.2 Tracking file ownership

Currently, the uninstall method will just consist of deleting all the files that were installed by a package. This brings about a problem: some files are provided by multiple packages. These should not be deleted upon the removal of one package that provided them if another package that also provided them is still installed.

There is also a second, though less significant, problem in the removal of directories. These should only be removed if they will be empty once all files to be deleted are removed, to prevent another program's files

residing in the same directory being deleted. The approach qpkg takes is simply not deleting directories [8], and while this still works, it is not very elegant.

Both problems can be solved by tracking which packages provide each file.

I chose to use gdbm [19] for the database holding this information. It provides a quick, lightweight, NoSQL method to retrieve data, and is already installed in a default LFS system [20].

The pkgfiles array is populated inside install, and this updateshared procedure is called just prior to final installation in the final directory.

```
updateshared() {
    shareddir="/var/lib/itch/shared"
    dbdir="$shareddir/files.gdbm"

    # create dir and file if not already existing
    sudo mkdir -p "$shareddir"
    sudo touch "$dbdir"

    for file in ${pkgfiles[@]}; do
        gdbmout=$(echo "fetch $file" | sudo gdbmtool "$dbdir" 2>/dev/null)

        echo "store $file \"$gdbmout $pkgname\"" | sudo gdbmtool "$dbdir"
    done
}
```

This uses gdbmtool (a command line interface to gdbm) to add the name of the installed package onto the list of packages that provide a certain file. This table maps each installed file (the key) to this list (the value).

The database is located at /var/lib/itch/shared/files.gdbm. It is in a "shared" directory to show that this file is used by all packages.

However, upon testing, it would take minutes for this procedure to run for larger packages. This was since every time gdbmtool is called (twice per file), the database is opened and closed, and while the actual reading and writing is very quick, this overhead greatly slowed down the process.

Therefore, I replaced this procedure with a short python script - updatedb.py - performing the same task.

```
import sys
import dbm.gnu

dbdir = sys.argv[1]
pkgname = sys.argv[2]

db = dbm.gnu.open(dbdir, 'c');

for i in range(3, len(sys.argv)):
    file = sys.argv[i]

    gdbmout = db.get(file)

    if gdbmout is not None:
        pkgs = gdbmout.decode().split(' ')
    else:
        pkgs = []

```

```
        # don't add pkgname if already existing - eg for reinstallations
        if pkgname not in pkgs:
            pkgs.append(pkgname)

        db[file] = " ".join(pkgs)

    db.close()
```

Python - unlike bash, which is only used for scripting - provides a well-documented interface to gdbm [21] such that the database can be opened once at the start of the program, and only closed at the end. This greatly reduced the time taken to update the database to milliseconds, from minutes.

This script is executed from inside the main bash file as follows:

```
    printf "%s" "${pkgfiles[@]}"
        | sudo python "$pythonupdatedir" "$filedbdir" "$pkgname"
```

The location of the database and package name are passed to it as command line arguments, while the package files are piped into it, as otherwise the number of files would often exceed the maximum length of command line arguments.

3.3 Uninstalling packages

The only reason such lengths have been taken to install packages, rather than simply issuing the installation command

```
    # sudo make install
```

is to ensure that these packages can be cleanly uninstalled, leaving no traces behind that could cause problems for other packages, or newer versions of the same package.

Before attempting to uninstall a package, it must first be verified that it is installed in the first place. The contents of /var/lib/itch/pkgs/ cannot be used for this, as they simply contain all archives that have been created. One may have been created but not installed, or installed but later uninstalled. The .itchpkg archives persist here intentionally, to allow easier installation again, but this does mean another method for tracking installed *packages* (not files) is needed.

I decided to also use gdbm for this, linking each package to its installed version.

```
    echo "store $pkgname_name $pkgname_version"
        | sudo gdbmtool "$pkgdbdir" 2>/dev/null
```

This does not require a separate python script, as the database is only accessed once per package installed.

The database is stored in /var/lib/itch/shared/pkgs.gdbm, again because this is shared between all packages.

```
    shareddir="/var/lib/itch/shared"
    pkgdbdir="$shareddir/pkgs.gdbm"
```

Arch Linux's pacman stores this data in the form of a tarball containing each package as a directory [2], however I chose to use this gdbm database instead since the hashing provided, and not needing to unpack the archive every time, makes it quicker to check whether a package is installed.

Having built this database, I then wrote the uninstallation procedure.

```
uninstall() {
    gdbmout="$(echo "fetch $pkgname" | sudo gdbmtool "$pkgdbdir" 2>/dev/null)"

    if [ "$gdbmout" = "" ]; then
        echo "Error! Package not installed to begin with!"
        exit
    fi

    pkgfiles="$(tar tzf "$itchpkgpath")"

    toremove="$(printf "%s" "${pkgfiles[@]}"
        | sudo python
            "$pythonuninstalldir" "$filedbdir" "$pkgname" "$itchpkgpath")"

    echo "$toremove"

    IFS=$'\n' read -rd '' -a toremovearr <<< "$toremove"

    for file in "${toremovearr[@]}"; do
        echo "$file"
        sudo rm -rf "$installprefix/$file"
    done

    pkgname_name="${pkgname%%-[0-9]*}"
    echo "delete $pkgname_name" | sudo gdbmtool "$pkgdbdir" 2>/dev/null
}
```

After confirming that the package is installed, the package's files are taken from the .itchpkg archive made in /var/lib/itch/pkgs/{pakage name} during installation. At this point, the python script uninstalldb.py is executed.

```
import sys
import dbm.gnu

dbdir = sys.argv[1]
pkgname = sys.argv[2]
itchpkgpath = sys.argv[3]

pkgfiles = [line.strip() for line in sys.stdin]

db = dbm.gnu.open(dbdir, 'c');

for file in pkgfiles:
    gdbmout = db.get(file)

    if gdbmout is not None:
        pkgs = gdbmout.decode().split(' ')
    else: # must have been an error
        pkgs = []
        continue

    pkgs.remove(pkgname)

    if len(pkgs) == 0:
        print(file)
```

```
            del db[file]
        else:
            db[file] = " ".join(pkgs)

    db.close()
```

This opens the files.gdbm database, and removes all occurrences of package being uninstalled. It also prints any files that were only provided by this package, so they can be captured by the main bash procedure, and deletes those entries from the database.

These files that are returned are then all deleted, and the package is removed from the pkgs.gdbm database, as it is no longer installed.

4 Programming the dependency resolver

4.1 Storing metadata

As previously established, there needs to be a database with multiple versions of each available package, storing dependency information. Here, I will specify the format in which this is stored.

Firstly, there will be a gdbm database linking each package name with its available versions (/var/lib/itch-resolver/global.gdbm).

The data for each of these package and version is stored in the general directory: /var/lib/itch-resolver/pkgs/{package name}/{package version}. This directory will contain three files: conflicts.itch, depends.itch, and provides.itch.

Together, these three files contain all the required information to resolve dependencies and output a combination of packages that all will work together. This format is easy to parse in a program, and as compact as possible, minimising the storage size needed while holding all of the necessary information.

4.1.1 conflicts.itch

Each line of this file is in the format:

{conflicting package name} {lowest conflicting version} {highest conflicting version}.

The lowest version can be 0, and highest can be left out if not applicable.

These are the packages and versions that cannot be installed together with this package.

4.1.2 depends.itch

This has a very similar format. Each package that is required for the target package has a line in the format:

{required package name} {lowest version required} {highest acceptable version}.

4.1.3 provides.itch

This stores programs that are provided by a certain package, and may be required by other packages. This file has the even simpler format:

{provided program name} {provided program version}.

4.2 Defining the SAT problem

A dependency resolution problem is expressed as a set of constraints: one package requires another to run, two package versions cannot co-exist, a certain package must be newer than a given version to support a specific version of another package.

These can be encoded as a boolean satisfiability (SAT) problem. This is clear, since both the SAT problem, and dependency resolution, are NP-complete, meaning they reduce to each other in at most polynomial time [11]. This is useful, since there exists programs called SAT solvers, which are highly optimised to compute large SAT problems.

Therefore, by encoding all the dependency constraints into conjunctive normal form (CNF - the format accepted by SAT solvers), the problem can be given to a SAT solver to find a satisfying arrangement of packages.

I will pass this generated CNF to MiniSAT [22], a popular, powerful but lightweight SAT solver with a well-documented shell interface [23].

The following rules will be encoded [24]:

1. A package's dependencies must be met, or the package not installed.
2. None of a package's conflicts should be installed, or the package not installed.
3. Only one version of any package can be installed.
4. Each package already installed on the system must remain installed in some version (this does not have to be the current one).
5. The package being installed should be installed with the given version.

4.2.1 The format of DIMACS CNF

CNF, or conjunctive normal form, is a format for expressing a boolean expression, where variables (which can only take true/false values) are expressed in a *conjunction* of *disjunctions*. For a disjunction to evaluate to true, at least one variable inside must be true. For a conjunction to do the same, *every* variable inside it must be true. Practically, this means CNF contains multiple lists of variables, in each of which at least one must be true in a satisfiable state. Variables can also be negated: this means that a false variable maps to true, and vice versa [25].

MiniSAT takes input of a CNF file in DIMACS format. DIMACS is a file format for representing CNF, where each line contains a list of integers (which correspond to the boolean variables in disjunction), with the lines in the file are in conjunction together. A negative integer encodes the negation of a variable. All lines must also end with an 0, but this is not a problem variable.

4.3 Generating a CNF file

First, we must create a mapping between each package-version and an integer, which will represent the package in the DIMACS file.

```
count = 1
for pkg in pkgs:
    versions = global_db[pkg.decode()].decode().split()

    for version in versions:
        pkg_to_num[pkg.decode(), version] = str(count)
        num_to_pkg[count] = (pkg.decode(), version)

        count += 1

    set_provides(pkg.decode(), version)
```

The provides data for the packages were also read and stored in this loop, so that they are finished before the conflicts are encoded (if p_a conflicts with p_b, and p_c provides p_b, then p_a also conflicts with p_c, but there is no way of knowing this before the provides data has been encoded.

At this point, we can also encode the dependency and conflict information for the packages.

```
for pkg in pkgs: # needs second loop so mappings (and provides) are complete!
    versions = global_db[pkg.decode()].decode().split()

    for version in versions:
        encode_deps(pkg.decode(), version)
        encode_conflicts(pkg.decode(), version)

    # encode "only one version of each package"
    for i in range(0, len(versions)):
        for j in range(i+1, len(versions)):
            sat_clauses.append("-" + pkg_to_num[pkg.decode(), versions[i]] +  " "
                               + "-" + pkg_to_num[pkg.decode(), versions[j]] + " 0")
```

Then, the required clauses for local packages are installed, ensuring the packages that are installed remain installed in some version.

```
local_pkgs_db = dbm.gnu.open(local_pkgs_db_dir, 'c')
local_pkgs = local_pkgs_db.keys()

full_closure = set()

for pkg in local_pkgs:
    # encode "each package installed should be installed (but whatever version)"
    versions = global_db[pkg.decode()].decode().split() # all available versions

    new_clause = ""
    for version in versions:
        new_clause += pkg_to_num[pkg.decode(), version] + " "

        full_closure.update(get_closure(pkg.decode(), version)) # get closure of each
    installed/installable pkg-ver
    new_clause += "0"

```

```
    sat_clauses.append(new_clause)

full_closure.update(get_closure(new_pkg_name, new_pkg_version))
```

Above, these packages are also added to a set representing a closure. This closure is expanded by the get_closure method to contain every dependency of the local packages, and all dependencies of those dependencies, and the dependencies of those, etc. This is also done for the package being installed.

With these, the program adds negated clauses for all packages that do not appear in the closure, ensuring they are not installed. This fixes an issue that came up during testing, where the SAT solver would arbitrarily define unnecessary packages as true. While this would result in a valid system state, the user should only be prompted to install the dependencies they need, rather than all of the packages that could coexist on their system.

```
for pkg_num, (pkg, version) in num_to_pkg.items():
    if (pkg, version) not in full_closure:
        sat_clauses.append("-" + str(pkg_num) + " 0")
```

The program terminates by printing all of the SAT clauses to standard output, and storing the mappings of DIMACS integers to packages in the closure in a json file.

```
for clause in sat_clauses:
    print(clause)

num_to_pkg_closure = dict()

for (pkg, ver) in full_closure:
    num_to_pkg_closure[pkg_to_num[pkg, ver]] = (pkg, ver)

with open(sys.argv[5], 'w') as file:
    json.dump(num_to_pkg_closure, file)
```

4.4 Full dependency resolution

The CNF-generating program defined above needs to be integrated with a MiniSAT to perform the actual resolutions and output a list of package-versions to be installed.

```
mkdir -p "$tempdir"

sudo python "$installdir/make-cnf.py" "$pkgsdir/" "$itchdir/" "$1" "$2"
    "$tempdir/mappings.json" > "$tempdir/output" # inputs are pkg and version
minisat "$tempdir/output" "$tempdir/minisat_out" > /dev/null # don't print sat solver output
(grep -oE '(^| )[0-9]+' "$tempdir/minisat_out" | tr -d ' ') > "$tempdir/positives"

python "$installdir/get-positives.py" "$tempdir/mappings.json" "$tempdir/positives"

rm -rf "$tempdir"
```

This bash script redirects the output of the make-cnf script into a temporary output file, passes that file to MiniSAT, which generates a list of all variables (representing packages) and whether they are true or false (representing to be installed or not). This file is trimmed to only include the positive, true, variables (packages to be installed), and this is passed to another python script that uses this list, as well as the json file mapping variables to package-versions, to print the complete list of packages that should be installed on the system to provide a minimal, unproblematic state. That script is below:

```
import json
import sys

print("Suggested system state [name, version]:\n")

with open(sys.argv[1], 'r') as file:
    num_to_pkg = json.load(file)

with open(sys.argv[2], 'r') as file:
    for line in file:
        line = line.rstrip() # remove trailing \n

        if line == "0":
            break

        print(num_to_pkg[line])
```

Note that all temporary files are placed in a specific directory, which is deleted upon the program's termination, so that unnecessary space is not taken up on the user's system.

5 The final product

5.1 Configuration files

During development, I hardcoded the file paths of the resources that need to be accessed by the various programs. However, this is not suitable for distribution, as users should be able to modify these variables in one place and have it take effect across all program files, and it is best practice to ensure users do not need to modify source code files.

Therefore, I created two configuration files, one for the installer, and the other for the resolver, holding these values. A typical setup (which is being used on my system) is shown below.

```
installprefix="/usr"
tempdir="/tmp/itch/build"

pkgpath="/var/lib/itch/pkgs"
shareddir="/var/lib/itch/shared"

itchinstallerdir="/usr/lib/itch-installer"
```

```
installdir="/usr/lib/itch-resolver"

pkgsdir="/var/lib/itch-resolver"

itchdir="/var/lib/itch"

tempdir="/tmp/itch-resolver"
```

These values are loaded by the relevant bash scripts as follows, based on the configuration file needed:

```
source "/etc/itch/itch.conf"
```

```
source "/etc/itch/itch-resolver.conf"
```

Note that the installprefix is where packages installed by the installer will be located (/usr is the default for Linux From Scratch systems).

Also note that the dependency resolver's configuration file contains an entry for itchdir, the directory where the files of the package installer are contained. This is not required, and if that directory is empty the dependency resolver will simply assume that no packages are currently installed on the system. Otherwise, if it is provided, it allows the dependency resolver to ensure already installed packages do not conflict with the one being installer, and vice-versa.

5.2 Typical directory tree

This diagram shows where all of the files associated with both the installer and dependency resolver are to be installed on a typical system. The configuration files allow scope for changing some of these, however this layout follows the Linux Filesystem Hierarchy Standard [16] and is therefore recommended.

This clearly shows the separation between the installer and resolver components, since they rely on and use separate directories.

```
/var/lib/
├── itch/
│   ├── pkgs/
│   │   └── [package name]-[package version]/
│   │       └── [package name].itchpkg
│   └── shared/
│       ├── files.gdbm
│       └── pkgs.gdbm
└── itch-resolver/
    ├── pkgs/
    │   └── [package name]/
    │       └── [package version]/
    │           ├── depends.itch
    │           ├── conflicts.itch
    │           └── provides.itch
    └── global.gdbm
/tmp/
├── itch/
│   └── build/
└── itch-resolver/
/usr/
├── bin/
│   ├── itch-installer
│   └── itch-resolver
└── lib/
    ├── itch-installer/
    │   ├── uninstalldb.py
    │   └── updatedb.py
    └── itch-resolver/
        ├── get-positives.py
        └── make-cnf.py
/etc/
└── itch/
    ├── itch.conf
    └── itch-resolver.conf
```

The package archives, storing installation information for future removal, and providing precompiled archives for reinstallation, are found under /var/lib/itch/pkgs/, and /var/lib/itch/shared/ contains information of the packages currently installed on the system, and the files that are installed (and which package(s) they belong to).

The dependency information is held at /var/lib/itch-resolver/. The depends, conflicts, and provides files are stored in the pkgs/ subdirectory, and the global.gdbm database holds information about what packages, and in what versions, the system has information about.

The temporary directory /tmp/itch/build/ is used as the fakeroot location for redirecting the installation of a package, and /tmp/itch-resolver/ stores the temporary files created while the dependency resolver runs.

The /usr/bin/ directory contains the executable shell scripts for both programs, and /usr/lib/ contains the python scripts they call during their runtime.

Finally, /etc/itch/ contains the configuration files for both programs. They are not in the users .config directory, since itch is designed to work system-wide.

5.3 Installation

Installation is generally simple, but has to be done separately for the package installer and dependency resolver components. This is by design, so that users can install only one, but not the other, if desired.

As dependencies, both components require Bash, Python 3, and GDBM. Additionally, the dependency resolver requires MiniSAT. MiniSAT is the only dependency that is not installed in a default Linux From Scratch installation.

This software has only been tested on a Linux From Scratch and an Arch Linux system. There is no reason that it should not also work on other distributions, but it is only intended to be used on Linux From Scratch, and so some subtleties of other systems may unexpectedly interact with the programs' functionality.

5.3.1 The package installer

The package installer will come with three files: itch-installer, uninstalldb.py, and updatedb.py.

itch-installer should be copied into /usr/bin/, and the two python scripts into /usr/lib/itch-installer/. This directory should be made if not already present.

The configuration file should be created in /etc/itch/itch.conf, containing the fields specified in Section 5.1.

Assuming the three files are in the current directory, the following script automates this process:

```
sudo cp itch-installer /usr/bin/

sudo mkdir -p /usr/lib/itch-installer/
sudo cp uninstalldb.py /usr/lib/itch-installer/
sudo cp updatedb.py /usr/lib/itch-installer/

sudo echo "installprefix="/usr"
tempdir="/tmp/itch/build"

pkgpath="/var/lib/itch/pkgs"
```

```
shareddir="/var/lib/itch/shared"

itchinstallerdir="/usr/lib/itch-installer"" > /etc/itch/itch.conf
```

5.3.2 The dependency resolver

The dependency resolver will come with three files: itch-resolver, make-cnf.py, and get-positives.py.

itch-resolver should be copied into /usr/bin/, and the two python scripts into /usr/lib/itch-resolver/. This directory should be made if not already present.

The configuration file should be created in /etc/itch/itch-resolver.conf, containing the fields specified in Section 5.1.

Assuming the three files are in the current directory, the following script automates this process:

```
sudo cp itch-resolver /usr/bin/

sudo mkdir -p /usr/lib/itch-resolver/
sudo cp make-cnf.py /usr/lib/itch-resolver/
sudo cp get-positives.py /usr/lib/itch-resolver/

sudo echo "installdir="/usr/lib/itch-resolver"

pkgsdir="/home/shayan/prog/school/epq/debian-pkgs"

itchdir="/var/lib/itch"

tempdir="/tmp/itch-resolver"" > /etc/itch/itch-resolver.conf
```

5.4 Example usage

Using Debian repo...

5.5 Conclusion

successes and failures of the package manager

what works, what doesn't

scope for improvements

establishing a basic/sophisticated level of success

evaluation against 7 goals

A Package installer

A.1 itch-installer

```
#!/bin/bash

source "/etc/itch/itch.conf"

filedbdir="$shareddir/files.gdbm"
pkgdbdir="$shareddir/pkgs.gdbm"

pythonupdatedir="$itchinstallerdir/updatedb.py"
pythonuninstalldir="$itchinstallerdir/uninstalldb.py"

configure() {
    echo "configure as normal, ensuring the following is added:"
    echo "    --prefix=$installprefix"
}

build() {
    echo "make as normal"
}

install() {
    rm -rf "$tempdir" # in case not properly cleaned before
    mkdir -p $tempdir

    make DESTDIR="$tempdir" install # install in temp dir

    findname

    sudo mkdir -p "$pkgpath/$pkgname"

    sudo tar czf "$itchpkgpath" -C "$tempdir$installprefix" .

    installpkg
}

installpkg() { # existing tarball in pkgpath
    cd "$installprefix"

    pkgfiles=$(tar tzf "$itchpkgpath")

    # finds existing files that will be overwritten - credit
    https://github.com/skeeto/dotfiles/blob/master/bin/qpkg (with significant modifications)
    for file in $(tar tzf "$itchpkgpath" | grep -v '/$'); do
        if [ -e "$file" ]; then
            printf "Overwrites '%s'\n" "${file#./}"
        fi
    done

    read -p "Proceed? (yes/NO): " confirmation

    if [ "${confirmation,,}" = "yes" ] || [ "${confirmation,,}" = "y" ]; then # ${[string],,}
    converts to lower case
```

```
50          updateshared
51          tar xf "$itchpkgpath" # files extracted into install dir
52      else
53          echo "Aborted!"
54      fi
55
56      rm -rf "$tempdir"
57  }
58
59  uninstall() {
60      pkgname_name="${pkgname%%-[0-9]*}"
61
62      gdbmout="$(echo "fetch $pkgname_name" | sudo gdbmtool "$pkgdbdir" 2>/dev/null)" # empty if
        not existing
63
64      if [ "$gdbmout" = "" ]; then # CHECK THIS LINE, NEW!
65          echo "Error! Package not installed to begin with!"
66          exit
67      fi
68
69      pkgfiles="$(tar tzf "$itchpkgpath")"
70
71      toremove="$(printf "%s" "${pkgfiles[@]}" | sudo python "$pythonuninstalldir" "$filedbdir"
        "$pkgname" "$itchpkgpath")"
72
73      IFS=$'\n' read -rd '' -a toremovearr <<< "$toremove"
74
75      for file in "${toremovearr[@]}"; do
76          echo "$file"
77          sudo rm -rf "$installprefix/$file"
78      done
79
80      echo "delete $pkgname_name" | sudo gdbmtool "$pkgdbdir" 2>/dev/null
81  }
82
83  findname() {
84      pkgname=$(basename "$(pwd)")
85
86      # tries to find package name (either current dir or parent if in build dir)
87      if [ "$pkgname" = "build" ]; then
88          pkgname=$(basename "$(dirname "$(pwd)")")
89      fi
90
91      echo "Enter package name (blank for [$pkgname]):"
92      read input
93
94      if [ "$input" != "" ]; then
95          pkgname="$input"
96      fi
97
98      itchpkgpath="$pkgpath/$pkgname/$pkgname.itchpkg"
99  }
100
101 getnamepkgd() { # get name for a pkg that has been packaged into a .itchpkg
102     pkgname="$2"
103
```

```
    itchpkgpath="$pkgpath/$pkgname/$pkgname.itchpkg"

    if [ ! -f "$itchpkgpath" ]; then
        echo "Error! Package archive not found!"
        exit
    fi
}

updateshared() {
    # create dir if not already existing | python handles creation of non-existent db
    sudo mkdir -p "$shareddir"

    # root due to location of file made
    printf "%s" "${pkgfiles[@]}" | sudo python "$pythonupdatedir" "$filedbdir" "$pkgname"

    pkgname_name="${pkgname%%-[0-9]*}"
    pkgname_version="${pkgname#${pkgname_name}-}"

    # define the pkg as installed with version given
    echo "store $pkgname_name $pkgname_version" | sudo gdbmtool "$pkgdbdir" 2>/dev/null
}

case "$1" in
    "-c")
        configure ;;
    "-b")
        build ;;
    "-i")
        if [ -z "$2" ]; then
            install
        else
            getnamepkgd "$@"
            installpkg
        fi ;;
    "-u")
        if [ -z "$2" ]; then
            echo "Error!"
        else
            getnamepkgd "$@"
            uninstall "$@"
        fi ;;
    *)
        echo "Error!"
        ;;
esac
```

A.2 updatedb.py

```
import sys
import dbm.gnu

dbdir = sys.argv[1]
pkgname = sys.argv[2]

```

```
pkgfiles = [line.strip() for line in sys.stdin]

db = dbm.gnu.open(dbdir, 'c');

for file in pkgfiles:
    gdbmout = db.get(file)

    if gdbmout is not None:
        pkgs = gdbmout.decode().split(' ')
    else:
        pkgs = []

    # don't add pkgname if already existing - eg for reinstallations
    if pkgname not in pkgs:
        pkgs.append(pkgname)

    db[file] = " ".join(pkgs)

db.close()
```

A.3 uninstalldb.py

```
import sys
import dbm.gnu

dbdir = sys.argv[1]
pkgname = sys.argv[2]
itchpkgpath = sys.argv[3]

pkgfiles = [line.strip() for line in sys.stdin]

db = dbm.gnu.open(dbdir, 'c');

for file in pkgfiles:
    gdbmout = db.get(file)

    if gdbmout is not None:
        pkgs = gdbmout.decode().split(' ')
    else: # must have been an error
        pkgs = []
        continue

    pkgs.remove(pkgname)

    if len(pkgs) == 0:
        print(file)
        del db[file]
    else:
        db[file] = " ".join(pkgs)

db.close()
```

B Dependency resolver

B.1 itch-resolver

```
#!/bin/bash

source "/etc/itch/itch-resolver.conf"

mkdir -p "$tempdir"

sudo python "$installdir/make-cnf.py" "$pkgsdir/" "$itchdir/" "$1" "$2"
    "$tempdir/mappings.json" > "$tempdir/output" # inputs are pkg and version
minisat "$tempdir/output" "$tempdir/minisat_out" > /dev/null # don't print sat solver output
(grep -oE '(^| )[0-9]+' "$tempdir/minisat_out" | tr -d ' ') > "$tempdir/positives"

python "$installdir/get-positives.py" "$tempdir/mappings.json" "$tempdir/positives"

rm -rf "$tempdir"
```

B.2 make-cnf.py

```
import sys
import dbm.gnu
import re
import os
from pathlib import Path
import json

core_dir = sys.argv[1] # /var/lib/itch-resolver/
global_db_dir = core_dir + "global.gdbm"
pkgs_dir = core_dir + "pkgs/"

local_pkgs_core_dir = sys.argv[2] # /var/lib/itch/
local_pkgs_db_dir = local_pkgs_core_dir + "shared/pkgs.gdbm" # /var/lib/itch/shared/pkgs.gdbm
local_pkgs_dir = local_pkgs_core_dir + "pkgs/"

new_pkg_name = sys.argv[3]
new_pkg_version = sys.argv[4] # NOTE: new pkg MUST be in global db

global_db = dbm.gnu.open(global_db_dir, 'c')

sat_clauses = list()

# mapping of pkgs to CNF number
pkg_to_num = dict()
num_to_pkg = dict()

pkg_to_deps = dict()
pkg_to_versions = dict()
pkg_to_deps = dict()
pkg_to_conflicts = dict()
pkg_to_provides = dict()

```

```
def build_pkg_dicts():
    return

def get_versions(pkg):
    dir = pkgs_dir + pkg

    if pkg in pkg_to_versions:
        return pkg_to_versions[pkg]

    if not Path(dir).exists():
        return []

    versions = os.listdir(dir)

    pkg_to_versions[pkg] = versions

    return versions

def version_in_range(version, lower, upper):
    if compare_versions(version, lower) == "<":
        return False
    if upper and compare_versions(version, upper) == ">":
        return False

    return True

def compare_versions(version_1, version_2):
    if version_1 == version_2:
        return "="

    v1a, v1b, v1c = split_version(version_1)
    v2a, v2b, v2c = split_version(version_2)

    if v1a > v2a:
        return ">"
    if v1a < v2a:
        return "<"

    for i in range(0, min(len(v1b), len(v2b))):
        if v1b[i] > v2b[i]:
            return ">"
        if v1b[i] < v2b[i]:
            return "<"

    if len(v1b) > len(v2b):
        return ">"
    if len(v1b) < len(v2b):
        return "<"

    if v1c > v2c:
        return ">"
    if v1c < v2c:
        return "<"

    return "="

```

```
89  def split_version(version):
90      if ":" in version:
91          epoch_str, version = version.split(":", 1)
92          epoch = int(epoch_str)
93      else:
94          epoch = 0
95
96      if "+" in version:
97          version = version.split("+", 1)[0]
98
99      if "-" in version:
100         main_str, end_str = version.rsplit("-", 1) # rsplit gets last occurance
101     else:
102         main_str, end_str = version, "0"
103
104     # split into array by .
105     main = [int(x) for x in main_str.split(".") if x.isnumeric()]
106
107     # strip text at end of c - only present in debian files
108     m = re.match(r"(\\d+)", end_str)
109     end = int(m.group(1)) if m else 0
110
111     return epoch, main, end
112
113
114 # encode "dependencies must be met, or the package not be installed"
115 def encode_deps(pkg, version):
116     dep_to_versions = get_deps(pkg, version)
117
118     for dep, vers in dep_to_versions.items():
119         new_clause = "-" + pkg_to_num[pkg, version] + " "
120
121         entered = False
122
123         for ver in vers:
124             entered = True
125             new_clause += pkg_to_num[dep, ver] + " "
126
127         new_clause += "0"
128
129         if (entered):
130             sat_clauses.append(new_clause)
131
132 # encode "conflicts must not exist, or the package not be installed"
133 def encode_conflicts(pkg, version):
134     conflict_to_versions = get_conflicts(pkg, version)
135
136     for dep, vers in conflict_to_versions.items():
137         new_clause = "-" + pkg_to_num[pkg, version] + " "
138
139         entered = False
140
141         for ver in vers:
142             entered = True
143
144             if (dep, ver) in provides:
```

```
                for providing_pkg, providing_ver in provides[dep, ver]:
                    new_clause += "-" + pkg_to_num[providing_pkg, providing_ver] + " "
            else:
                new_clause += "-" + pkg_to_num[dep, ver] + " " # not pkg or not conflict

        new_clause += "0"

        if (entered):
            sat_clauses.append(new_clause)

def get_deps(pkg, version):
    pkg_dir = pkgs_dir + pkg + "/" + version + "/"
    dep_file = pkg_dir + "depends.itch"

    if (pkg, version) in pkg_to_deps:
        return pkg_to_deps[pkg, version]

    results = dict()

    if not Path(dep_file).exists():
        pkg_to_deps[pkg, version] = results
        return results

    with open(dep_file) as file:
        for line in file:
            fields = line.strip().split()

            dep_name = fields[0]
            lower = fields[1]
            upper = ""

            if len(fields) > 2:
                upper = fields[2]

            dep_versions = []

            for ver in get_versions(dep_name):
                if version_in_range(ver, lower, upper):
                    dep_versions.append(ver)

            results[dep_name] = dep_versions # maps dependency name -> list of allowed versions

    pkg_to_deps[pkg, version] = results

    return results

def get_conflicts(pkg, version):
    pkg_dir = pkgs_dir + pkg + "/" + version + "/"
    conflict_file = pkg_dir + "conflicts.itch"

    results = dict()

    if not Path(conflict_file).exists():
        return results

    with open(conflict_file) as file:
```

```
        for line in file:
            fields = line.strip().split()

            conflict_name = fields[0]
            lower = fields[1]
            upper = ""

            if len(fields) > 2:
                upper = fields[2]

            conflict_versions = []

            for ver in get_versions(conflict_name):
                if version_in_range(ver, lower, upper):
                    conflict_versions.append(ver)

            results[conflict_name] = conflict_versions # maps conflict name -> list of not allowed versions

    return results

def set_provides(pkg, version):
    pkg_dir = pkgs_dir + pkg + "/" + version + "/"
    provide_file = pkg_dir + "provides.itch"

    if not Path(provide_file).exists():
        return

    with open(provide_file) as file:
        for line in file:
            provided_pkg, provided_ver = line.strip().split()

            if (provided_pkg, provided_ver) in provides:
                provides[provided_pkg, provided_ver].append((pkg, version))
            else:
                provides[provided_pkg, provided_ver] = [(pkg, version)]

def get_closure(pkg, ver):
    start = (pkg, ver)
    queue = [start]
    visited = set()

    while len(queue) > 0:
        (p, v) = queue.pop()

        if (p, v) in visited:
            continue

        visited.add((p, v))

        deps = get_deps(p, v)
        for dep in deps.keys():
            for allowed_ver in deps[dep]:
                queue.append((dep, allowed_ver))

    return visited
```

```
256
257
258 pkgs = global_db.keys()
259 provides = dict() # mapping of (provided pkg, provided pkg version) -> list of (pkg, version)
260
261 count = 1
262 for pkg in pkgs:
263     versions = global_db[pkg.decode()].decode().split()
264
265     for version in versions:
266         pkg_to_num[pkg.decode(), version] = str(count)
267         num_to_pkg[count] = (pkg.decode(), version)
268
269         count += 1
270
271         set_provides(pkg.decode(), version)
272
273 for pkg in pkgs: # needs second loop so mappings (and provides) are complete!
274     versions = global_db[pkg.decode()].decode().split()
275
276     for version in versions:
277         encode_deps(pkg.decode(), version)
278         encode_conflicts(pkg.decode(), version)
279
280     # encode "only one version of each package"
281     for i in range(0, len(versions)):
282         for j in range(i+1, len(versions)):
283             sat_clauses.append("-" + pkg_to_num[pkg.decode(), versions[i]] +  " "
284                                + "-" + pkg_to_num[pkg.decode(), versions[j]] + " 0")
285
286 local_pkgs_db = dbm.gnu.open(local_pkgs_db_dir, 'c')
287
288 local_pkgs = local_pkgs_db.keys()
289
290 full_closure = set()
291
292 for pkg in local_pkgs:
293     # encode "each package installed should be installed (but whatever version)"
294     versions = global_db[pkg.decode()].decode().split() # all available versions
295
296     new_clause = ""
297     for version in versions:
298         new_clause += pkg_to_num[pkg.decode(), version] + " "
299
300         full_closure.update(get_closure(pkg.decode(), version)) # get closure of each
    installed/installable pkg-ver
301     new_clause += "0"
302
303     sat_clauses.append(new_clause)
304
305 full_closure.update(get_closure(new_pkg_name, new_pkg_version))
306
307 # encode "pkg to install should be installed with GIVEN version"
308 sat_clauses.append(pkg_to_num[new_pkg_name, new_pkg_version] + " 0")
309
310 for pkg_num, (pkg, version) in num_to_pkg.items():
```

```
    if (pkg, version) not in full_closure:
        sat_clauses.append("-" + str(pkg_num) + " 0")

for clause in sat_clauses:
    print(clause)

num_to_pkg_closure = dict()

for (pkg, ver) in full_closure:
    num_to_pkg_closure[pkg_to_num[pkg, ver]] = (pkg, ver)

with open(sys.argv[5], 'w') as file:
    json.dump(num_to_pkg_closure, file)
```

B.3 get-positives.py

```
import json
import sys

print("Suggested system state [name, version]:\n")

with open(sys.argv[1], 'r') as file:
    num_to_pkg = json.load(file)

with open(sys.argv[2], 'r') as file:
    for line in file:
        line = line.rstrip() # remove trailing \n

        if line == "0":
            break

        print(num_to_pkg[line])
```

References

[1] Richard Stallman. *Linux and the GNU System*. 2nd Nov. 2021. URL: https://www.gnu.org/gnu/linux-and-gnu.en.html (visited on 26/08/2025).

[2] Arch Linux contributors. *pacman*. 6th Sept. 2025. URL: https://wiki.archlinux.org/title/Pacman (visited on 07/09/2025).

[3] Linux From Scratch. *Welcome to Linux From Scratch!* 2025. URL: https://www.linuxfromscratch.org/ (visited on 24/08/2205).

[4] Linux From Scratch. *Package Management*. 5th Mar. 2025. URL: https://www.linuxfromscratch.org/lfs/view/stable/chapter08/pkgmgt.html (visited on 24/08/2025).

[5] Ben Van Daele. *Pacman on LFS 8.1*. 1st Aug. 2019. URL: https://github.com/benvd/lfs-pacman?tab=readme-ov-file (visited on 24/08/2025).

[6] Reddit. *I'm curious as to how many linux users can code*. 2022. URL: https://archive.is/2tcxS (visited on 07/09/2025).

[7] Tushar Teredesai. *Fakeroot approach for package installation*. 17th Jan. 2006. URL: https://www.linuxfromscratch.org/hints/downloads/files/fakeroot.txt (visited on 20/08/2025).

[8] Chris Wellons. *A Crude Personal Package Manager*. 27th Mar. 2018. URL: https://nullprogram.com/blog/2018/03/27/ (visited on 20/05/2025).

[9] Pierre Hebert. *TRIP, a TRIvial Packager for LFS (and other linux systems)*. 13th May 2006. URL: https://www.linuxfromscratch.org/hints/downloads/files/package_management_using_trip.txt (visited on 20/08/2025).

[10] NixOS contributors. *Why You Should Give it a Try*. URL: https://nixos.org/guides/nix-pills/01-why-you-should-give-it-a-try.html (visited on 18/08/2025).

[11] Pietro Abate et al. *Dependency Solving: a Separate Concern in Component Evolution Management*. Univ Paris Diderot and INRIA Paris-Rocquencourt, 2012.

[12] Arch Linux contributors. *Arch Linux*. 3rd Sept. 2025. URL: https://wiki.archlinux.org/title/Arch_Linux (visited on 20/09/2025).

[13] Debian contributors. *WhyDebian*. 3rd July 2022. URL: https://wiki.debian.org/WhyDebian#Utility_and_usability (visited on 20/09/2025).

[14] Gentoo contributors. *ebuild repository*. 29th May 2025. URL: https://wiki.gentoo.org/wiki/Ebuild_repository#The_Gentoo_ebuild_repository (visited on 21/09/2025).

[15] Mike Gancarz. "I. The Unix Philosophy". In: Dec. 2003. ISBN: 9781555582739. DOI: 10.1016/B978-155558273-9/50003-8.

[16] The Linux Foundation. *Filesystem Hierarchy Standard*. 19th Mar. 2015. URL: https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.pdf (visited on 10/09/2025).

[17] GNU. *GNU Tar*. 18th July 2023. URL: https://www.gnu.org/software/tar/ (visited on 10/09/2025).

[18] Chris Wellons. *qpkg*. 27th Jan. 2023. URL: https://github.com/skeeto/dotfiles/blob/master/bin/qpkg (visited on 20/05/2025).

[19] Pierre Gaumond et al. *GNU dbm*. 2025. URL: https://www.gnu.org.ua/software/gdbm/manual/gdbm.pdf (visited on 03/09/2025).

[20] Linux From Scratch. *GDBM-1.26*. 1st Sept. 2025. URL: https://www.linuxfromscratch.org/lfs/view/stable-systemd/chapter08/gdbm.html (visited on 03/09/2025).

[21] Python Software Foundation. *dbm — Interfaces to Unix "databases"*. 11th Sept. 2025. URL: https://docs.python.org/3/library/dbm.html#module-dbm.gnu (visited on 11/09/2025).

[22] Niklas Eén and Niklas Sörensson. “An Extensible SAT-solver”. In: *Theory and Applications of Satisfiability Testing*. Ed. by Enrico Giunchiglia and Armando Tacchella. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 502–518.

[23] David A. Wheeler. *MiniSAT User Guide*. 28th June 2008. URL: https://dwheeler.com/essays/minisat-user-guide.html (visited on 23/09/2025).

[24] Robert Atkey. *Package Installation Problem*. URL: https://personal.cis.strath.ac.uk/robert.atkey/cs208/packages.html (visited on 23/09/2025).

[25] John Burkardt. *CNF Files*. URL: https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html (visited on 02/10/2025).